● PROGETTO UNIVERSITARIO · A.A. 2025/2026

ESG-BALANCE

Piattaforma per la gestione di bilanci aziendali e indicatori ESG

Corso di Basi di Dati — Informatica per il Management
Università di Bologna

Tesfaye Venieri · Matricola 0000990286

PHP 8.2 | MySQL 8.0 | MongoDB 5.0 | Bootstrap 5.3 | Chart.js 4.4 | XAMPP

Panoramica del Progetto

ESG-BALANCE è un'applicazione web che permette di gestire i **bilanci aziendali** e collegarli agli **indicatori ESG** (Environmental, Social, Governance). Prevede tre ruoli utente e un processo di revisione con approvazione o respingimento.

🛡 Amministratore

- Gestisce template voci contabili
- Crea e categorizza indicatori ESG
- Assegna revisori ai bilanci
- Monitora statistiche globali

🔍 Revisore ESG

- Esamina i bilanci assegnati
- Inserisce note di revisione
- Emette giudizio finale
- Indice di affidabilità automatico

🏢 Responsabile

- Gestisce le proprie aziende
- Crea e compila bilanci annuali
- Collega indicatori ESG alle voci
- Monitora lo stato revisioni

WORKFLOW BILANCIO

FASE 1		FASE 2 — TRIGGER		ESITO A — TRIGGER		ESITO B — TRIGGER
Bozza	→	**In Revisione**	→	**Approvato**	→	**Respinto**

Schema Entity-Relationship

ENTITÀ PRINCIPALI

ENTITÀ	ATTRIBUTI CHIAVE
utenti	username PK, CF UNIQUE, ruolo ENUM
revisori	username FK, nr_revisioni, indice_affidabilita
responsabili	username FK, curriculum_pdf
aziende	id PK, ragione_sociale UNIQUE, partita_iva UNIQUE
bilanci	id PK, id_azienda FK, anno, stato ENUM
voci_contabili	nome PK, descrizione
indicatori_esg	nome PK, rilevanza, tipo ENUM

RELAZIONI E CARDINALITÀ

RELAZIONE	TIPO	TABELLA PONTE
utenti → revisori / responsabili	Specializ. T,E	—
responsabili — aziende	1:N	—
aziende — bilanci	1:N	—
bilanci — voci_contabili	N:M	valori_bilancio
bilanci — revisori	N:M	revisioni
indicatori_esg — voci_contabili	N:M	voci_indicatori
revisori — giudizi	1:N	—
revisori — note_revisione	1:N	—

ℹ **17 tabelle relazionali** in forma BCNF — ridondanza controllata solo su `nr_bilanci` (gestita da trigger T3/T4 per performance).

Analisi della Ridondanza — `nr_bilanci`

TAVOLA DEI VOLUMI

CONCETTO	TIPO	VOLUME
aziende	Entità	10
bilanci	Entità	50 (5 per azienda)

TAVOLA DELLE OPERAZIONI

#	OPERAZIONE	FREQ.	TIPO
OP1	Aggiungere azienda + 3 bilanci	1/mese	I
OP2	Contare bilanci per tutte le aziende	3/mese	B
OP3	Rimuovere azienda + bilanci connessi	1/mese	B

ℹ Parametri: **wI = 1**, **wB = 0.5**, **a = 2** (costo scrittura = 2 × lettura)

COSTO TOTALE MENSILE

OP.	CON RIDONDANZA	SENZA RIDONDANZA
OP1	$1W_{az} + 3W_{bil} + 3W_{az}$(T3) (2 + 6 + 6) × wI × 1 = **14**	$1W_{az} + 3W_{bil}$ (2 + 6) × wI × 1 = **8**
OP2	$10R_{az}$ (legge nr_bilanci) 10 × wB × 3 = **15**	$10R_{az} + 50R_{bil}$ (COUNT JOIN) 60 × wB × 3 = **90**
OP3	$1R+1W_{az} + 5W_{bil} + 5W_{az}$(T4) (3 + 10 + 10) × wB × 1 = **11.5**	$1R+1W_{az} + 5W_{bil}$ (cascade) (3 + 10) × wB × 1 = **6.5**
TOTALE	**40.5**	**104.5**

Decisione: si mantiene la ridondanza (40.5 < 104.5).
Il risparmio su OP2 (lettura batch 3×/mese) è determinante: la JOIN completa su 50 bilanci costa 6× di più rispetto alla semplice lettura del campo `nr_bilanci`. Il trigger T3/T4 gestisce l'aggiornamento automatico senza intervento applicativo.

DDL — Creazione Tabelle (estratto)

GERARCHIA UTENTI

SQL

```sql
CREATE TABLE utenti (
  username        VARCHAR(50) PRIMARY KEY,
  password_hash   VARCHAR(255) NOT NULL,
  codice_fiscale  CHAR(16) NOT NULL,
  data_nascita    DATE NOT NULL,
  luogo_nascita   VARCHAR(100) NOT NULL,
  ruolo
ENUM('amministratore','revisore','responsabile'),
  UNIQUE KEY (codice_fiscale)
);

CREATE TABLE revisori (
  username            VARCHAR(50) PRIMARY KEY,
  nr_revisioni        INT DEFAULT 0,
  indice_affidabilita DECIMAL(3,2) DEFAULT 0.00,
  FOREIGN KEY (username) REFERENCES utenti(username)
    ON DELETE CASCADE
);
```

AZIENDE E BILANCI

SQL

```sql
CREATE TABLE aziende (
  id                    INT AUTO_INCREMENT PRIMARY
KEY,
  ragione_sociale       VARCHAR(200) UNIQUE NOT
NULL,
  partita_iva           VARCHAR(11) UNIQUE NOT NULL,
  settore               VARCHAR(100),
  nr_bilanci            INT DEFAULT 0,
  username_responsabile VARCHAR(50),
  FOREIGN KEY (username_responsabile)
    REFERENCES responsabili(username)
);

CREATE TABLE bilanci (
  id             INT AUTO_INCREMENT PRIMARY KEY,
  id_azienda     INT NOT NULL,
  anno           YEAR NOT NULL,
  data_creazione DATE NOT NULL,
  stato
ENUM('bozza','in_revisione','approvato','respinto'),
  UNIQUE KEY (id_azienda, anno),
  FOREIGN KEY (id_azienda) REFERENCES aziende(id)
);
```

Trigger — 4 totali

T1 — CAMBIO STATO: BOZZA → IN REVISIONE

SQL

```sql
CREATE TRIGGER trg_bilancio_in_revisione
AFTER INSERT ON revisioni
FOR EACH ROW
BEGIN
  UPDATE bilanci
  SET stato = 'in_revisione'
  WHERE id = NEW.id_bilancio
    AND stato = 'bozza';
END;
```

T3/T4 — RIDONDANZA NR_BILANCI

SQL

```sql
-- T3: AFTER INSERT ON bilanci
CREATE TRIGGER trg_incrementa_nr_bilanci
AFTER INSERT ON bilanci FOR EACH ROW
BEGIN
  UPDATE aziende SET nr_bilanci = nr_bilanci + 1
  WHERE id = NEW.id_azienda;
END;

-- T4: AFTER DELETE ON bilanci
CREATE TRIGGER trg_decrementa_nr_bilanci
AFTER DELETE ON bilanci FOR EACH ROW
BEGIN
  UPDATE aziende SET nr_bilanci =
    GREATEST(nr_bilanci - 1, 0)
  WHERE id = OLD.id_azienda;
END;
```

T2 — CHIUSURA DOPO TUTTI I GIUDIZI

SQL

```sql
CREATE TRIGGER trg_bilancio_giudizio
AFTER INSERT ON giudizi
FOR EACH ROW
BEGIN
  DECLARE v_tot_revisori  INT;
  DECLARE v_tot_giudizi   INT;
  DECLARE v_respingimenti INT;

  SELECT COUNT(*) INTO v_tot_revisori
  FROM revisioni
  WHERE id_bilancio = NEW.id_bilancio;

  SELECT COUNT(*) INTO v_tot_giudizi
  FROM giudizi
  WHERE id_bilancio = NEW.id_bilancio;

  IF v_tot_giudizi = v_tot_revisori THEN
    SELECT COUNT(*) INTO v_respingimenti
    FROM giudizi
    WHERE id_bilancio = NEW.id_bilancio
      AND esito = 'respingimento';

    IF v_respingimenti > 0 THEN
      UPDATE bilanci SET stato='respinto'
      WHERE id = NEW.id_bilancio;
    ELSE
      UPDATE bilanci SET stato='approvato'
      WHERE id = NEW.id_bilancio;
    END IF;
  END IF;

  CALL sp_aggiorna_indice_affidabilita(
    NEW.username_revisore
  );
END;
```

Stored Procedures — 14 totali

SP1	SP2	SP3	SP4	SP5	SP6	SP7–14
LOGIN	REGISTRA	CREA AZIENDA	CREA BILANCIO	VOCE CONTABILE	VALORE VOCE	REVISIONE & ESG

SP2 — REGISTRAZIONE (TRANSAZIONE ATOMICA)

SQL

```sql
CREATE PROCEDURE sp_registra_utente(
  IN p_username VARCHAR(50), IN p_password_hash
VARCHAR(255),
  IN p_codice_fiscale CHAR(16), IN p_data_nascita
DATE,
  IN p_ruolo VARCHAR(20), IN p_curriculum_pdf
VARCHAR(255)
)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN ROLLBACK; RESIGNAL; END;
  START TRANSACTION;
  INSERT INTO utenti (...) VALUES (...);
  INSERT INTO email_utente (...) VALUES (...);
  IF p_ruolo = 'revisore' THEN
    INSERT INTO revisori (username) VALUES
(p_username);
  ELSEIF p_ruolo = 'responsabile' THEN
    INSERT INTO responsabili (...) VALUES (...);
  END IF;
  COMMIT;
END;
```

SP14 — AGGIORNAMENTO INDICE AFFIDABILITÀ

SQL

```sql
CREATE PROCEDURE sp_aggiorna_indice_affidabilita(
  IN p_username VARCHAR(50)
)
BEGIN
  DECLARE v_totale   INT DEFAULT 0;
  DECLARE v_positivi INT DEFAULT 0;

  SELECT COUNT(*) INTO v_totale
  FROM giudizi
  WHERE username_revisore = p_username;

  SELECT COUNT(*) INTO v_positivi
  FROM giudizi
  WHERE username_revisore = p_username
    AND esito = 'approvazione';

  IF v_totale > 0 THEN
    UPDATE revisori
    SET indice_affidabilita =
      ROUND(v_positivi / v_totale, 2)
    WHERE username = p_username;
  END IF;
END;
```

Query di Interrogazione

Q1 — BILANCI IN REVISIONE

SQL

```sql
SELECT b.id, a.ragione_sociale,
       b.anno, b.stato
FROM   bilanci b
JOIN   aziende a ON b.id_azienda = a.id
WHERE  b.stato = 'in_revisione'
ORDER BY b.data_creazione DESC;
```

Q2 — REVISORI CON COMPETENZE (LIVELLO ≥ 3)

SQL

```sql
SELECT cr.username,
       cr.nome_competenza,
       cr.livello
FROM   competenze_revisore cr
WHERE  cr.livello >= 3
ORDER BY cr.livello DESC;
```

Q3 — INDICATORI ESG AMBIENTALI COLLEGATI

SQL

```sql
SELECT ie.nome        AS indicatore,
       ie.tipo,
       vi.valore_indicatore,
       vi.nome_voce    AS voce
FROM   voci_indicatori vi
JOIN   indicatori_esg ie
       ON vi.nome_indicatore = ie.nome
WHERE  ie.tipo = 'ambientale';
```

Q4 — NOTE DI REVISIONE PER BILANCIO

SQL

```sql
SELECT u.codice_fiscale,
       nr.testo,
       nr.data_nota
FROM   note_revisione nr
JOIN   utenti u
       ON nr.username_revisore = u.username
WHERE  nr.id_bilancio = ?;
```

Viste — 4 totali

V1 / V2 — CONTATORI PIATTAFORMA

SQL

```sql
CREATE VIEW v_num_aziende AS
SELECT COUNT(*) AS totale_aziende
FROM aziende;

CREATE VIEW v_num_revisori AS
SELECT COUNT(*) AS totale_revisori
FROM revisori;
```

V4 — CLASSIFICA BILANCI PER INDICATORI ESG

SQL

```sql
CREATE VIEW v_classifica_bilanci_esg AS
SELECT b.id AS id_bilancio,
       a.nome AS azienda,
       b.anno, b.stato,
       COUNT(vi.nome_indicatore)
         AS num_indicatori_esg
FROM   bilanci b
JOIN   aziende a ON a.id = b.id_azienda
LEFT JOIN voci_indicatori vi
       ON vi.id_bilancio = b.id
GROUP BY b.id, a.nome, b.anno, b.stato
ORDER BY num_indicatori_esg DESC;
```

V3 — CLASSIFICA AFFIDABILITÀ AZIENDE

SQL

```sql
CREATE VIEW v_affidabilita_aziende AS
SELECT
  a.id AS id_azienda,
  a.nome, a.ragione_sociale,
  COUNT(DISTINCT b.id)
    AS bilanci_giudicati,
  COUNT(DISTINCT CASE WHEN NOT EXISTS (
    SELECT 1 FROM giudizi g2
    WHERE  g2.id_bilancio = b.id
      AND   g2.esito <> 'approvazione'
  ) THEN b.id ELSE NULL END)
    AS bilanci_approvati_puri,
  ROUND(
    COUNT(DISTINCT CASE WHEN ... END)
    * 100.0 / COUNT(DISTINCT b.id),
    2
  ) AS percentuale_affidabilita
FROM   aziende a
JOIN   bilanci b ON b.id_azienda = a.id
JOIN   giudizi g ON g.id_bilancio = b.id
GROUP BY a.id, a.nome, a.ragione_sociale
ORDER BY percentuale_affidabilita DESC;
```

Sicurezza e Applicazione Web

SICUREZZA APPLICATIVA

MISURA	DETTAGLIO
Password	bcrypt cost=12 via `password_hash()`
SQL Injection	PDO prepared statements al 100%
XSS	`htmlspecialchars()` su tutti gli output
CSRF	Token su ogni form, verificato con `hash_equals()`
Upload	Whitelist MIME + ext con `finfo`
Sessione	Timeout 1h, `httponly`, `samesite=Strict`
Errori	Solo `error_log()`, mai esposti all'utente

INTEGRITÀ DB

VINCOLO	DETTAGLIO
Foreign Keys	ON DELETE CASCADE su tabelle figlio
CHECK	rilevanza 0–10, livello competenza 0–5
Trigger	4 trigger per consistenza automatica stato
Transazioni	SP atomiche con ROLLBACK su SQLEXCEPTION

STACK TECNOLOGICO

LAYER	TECNOLOGIA
Backend	PHP 8.2 — procedurale + PDO
DB primario	MySQL 8.0 · 17 tabelle BCNF
DB logging	MongoDB 5.0 · log_eventi (con fallback MySQL)
Frontend	Bootstrap 5.3 · Inter font · Chart.js 4.4
Server	XAMPP · Apache 2.4

PAGINE PRINCIPALI

Admin

- template.php
- indicatori.php
- assegna_revisore.php

Revisore

- competenze.php
- revisione.php
- giudizio.php

Responsabile

- aziende.php
- bilancio.php
- indicatori_bilancio.php

Comune

- index.php (landing)
- login.php / register.php
- dashboard.php

Conclusioni

17	14	4	4
TABELLE BCNF	STORED PROCEDURES	TRIGGER	VISTE

DATABASE

- 17 tabelle relazionali in BCNF
- 14 stored procedures con transazioni
- 4 trigger per automazione stato
- 4 viste per statistiche e classifica
- Ridondanza controllata (`nr_bilanci`)
- MongoDB per event logging asincrono

APPLICAZIONE

- 3 ruoli utente distinti e isolati
- Autenticazione sicura (bcrypt + sessione)
- Workflow bilanci end-to-end
- Interfaccia responsive (Bootstrap 5.3)
- CSRF + validazione MIME upload
- Dashboard con grafici Chart.js

FUNZIONALITÀ

- Gestione bilanci annuali con voci
- Indicatori ESG (Ambientale, Sociale, Governance)
- Sistema revisione multi-revisore
- Note di revisione e giudizi con trigger
- Statistiche tramite viste SQL
- Indice affidabilità revisori automatico

> Progetto realizzato per l'esame di **Basi di Dati** — Corso di Informatica per il Management · Università di Bologna · A.A. 2025/2026 · **Tesfaye Venieri** — Matricola 0000990286